

## Practical No.: 01

**Aim:** To Implement advanced deep learning algorithms such as convolutional neural networks (CNNs) or recurrent neural networks (RNNs) using Python libraries like TensorFlow or PyTorch.

### Theory:

- Deep learning uses multi-layer neural networks to learn complex patterns from large datasets.
- CNNs are mainly used for images and extract features like edges, textures, and shapes using convolution filters.
- RNNs, including LSTM and GRU, are designed for sequential data such as text, speech, and signals.
- CNNs reduce spatial dimensions through pooling, while RNNs maintain memory across time steps.
- Libraries like TensorFlow and PyTorch simplify implementation with GPU acceleration and pre-trained modules.

**Requirements:** pip install torch torchvision torchaudio

### Code:

```
import torch

import torch.nn as nn

import torch.optim as optim

from torch.utils.data import DataLoader, Dataset

import numpy as np

# Custom Dataset for Random Sequences

class SequenceDataset(Dataset):

    def __init__(self, num_sequences=1000,
sequence_length=10, num_classes=2):

        self.num_sequences = num_sequences

        self.sequence_length = sequence_length

        self.num_classes = num_classes

        self.data = np.random.randn(num_sequences,
sequence_length, 1)

        self.labels = np.random.randint(0, num_classes,
num_sequences)
```

```
def __len__(self):
    return self.num_sequences

def __getitem__(self, idx):
    sequence = self.data[idx]
    label = self.labels[idx]
    return torch.tensor(sequence, dtype=torch.float32),
    torch.tensor(label, dtype=torch.long)

# LSTM Model Definition
class LSTMClassifier(nn.Module):
    def __init__(self, input_size=1, hidden_size=64,
num_layers=2, num_classes=2):
        super(LSTMClassifier, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.lstm = nn.LSTM(input_size, hidden_size,
num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        h0 = torch.zeros(self.num_layers, x.size(0),
self.hidden_size).to(x.device)
        c0 = torch.zeros(self.num_layers, x.size(0),
self.hidden_size).to(x.device)
        out, _ = self.lstm(x, (h0, c0))
        out = out[:, -1, :] # take output from the last
time step
        out = self.fc(out)
        return out

# Hyperparameters
input_size = 1
hidden_size = 64
num_layers = 2
```

```
num_classes = 2
num_epochs = 10
batch_size = 32
learning_rate = 0.001

# Dataset and DataLoader
dataset = SequenceDataset()
train_loader = DataLoader(dataset, batch_size=batch_size,
                           shuffle=True)

# Device configuration
device = torch.device("cuda" if torch.cuda.is_available()
                      else "cpu")

# Model, Loss, Optimizer
model = LSTMClassifier(input_size, hidden_size, num_layers,
                       num_classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# Training Loop
for epoch in range(num_epochs):
    for sequences, labels in train_loader:
        sequences, labels = sequences.to(device),
        labels.to(device)

        outputs = model(sequences)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f'Epoch [{epoch + 1}/{num_epochs}], Loss:
    {loss.item():.4f}')
```

```
# Test the model with a random sequence
with torch.no_grad():
    test_sequence = torch.randn(1, dataset.sequence_length,
input_size).to(device)
    prediction = model(test_sequence)
    predicted_class = torch.argmax(prediction, dim=1).item()
    print(f'Predicted class: {predicted_class}')
```

**Output:**

```
Epoch [1/10], Loss: 0.6809
Epoch [2/10], Loss: 0.6804
Epoch [3/10], Loss: 0.6799
Epoch [4/10], Loss: 0.6935
Epoch [5/10], Loss: 0.6939
Epoch [6/10], Loss: 0.6760
Epoch [7/10], Loss: 0.6860
Epoch [8/10], Loss: 0.7044
Epoch [9/10], Loss: 0.6657
Epoch [10/10], Loss: 0.6681
Predicted class: 0
```

## Practical No.: 02

**Aim:** To Build a Natural Language Processing (NLP) model for Sentiment analysis or text classification.

**Requirements:** pip install pandas scikit-learn datasets numpy

### Theory:

- NLP enables machines to understand human language for classification and analysis.
- Sentiment analysis identifies if text is positive, negative, or neutral.
- Text is converted to numbers using Bag-of-Words, TF-IDF, or Word Embeddings.
- ML models like Naive Bayes, SVM, and modern transformers (BERT, RoBERTa) provide high accuracy.
- Applications include product reviews, social media monitoring, and customer feedback systems.

### Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from datasets import load_dataset

# Load IMDB dataset from Hugging Face
print("Loading IMDB dataset...")
dataset = load_dataset("imdb")
print(dataset)

# Convert dataset to pandas DataFrame
df = pd.DataFrame({
    'text': dataset['train']['text'],
    'label': dataset['train']['label']
})
```

```
# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    df['text'], df['label'], test_size=0.2, random_state=42
)

# Convert text into TF-IDF features
vectorizer = TfidfVectorizer(stop_words='english',
                             max_features=5000)
X_train_vectorized = vectorizer.fit_transform(X_train)
X_test_vectorized = vectorizer.transform(X_test)

# Train logistic regression model
print("Training model...")
model = LogisticRegression(max_iter=1000)
model.fit(X_train_vectorized, y_train)

# Evaluate model
predictions = model.predict(X_test_vectorized)
accuracy = accuracy_score(y_test, predictions)
print(f'Accuracy: {accuracy * 100:.2f}%')

# Test with new sample text
new_text = ["I did not like this at all."]
new_vectorized = vectorizer.transform(new_text)
prediction = model.predict(new_vectorized)
print(f'Prediction for "{new_text[0]}": {"positive" if
prediction[0] == 1 else "negative"}')
```

**Output:**

```
Loading IMDB dataset...
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
Training model...
Accuracy: 88.40%
Prediction for "I did not like this at all.": negative
```

## Practical No.: 03

**Aim:** To Create a ChatBot using Advanced Techniques like Transformer models.

**Requirements:** pip install torch transformers datasets tokenizers accelerate

### Theory:

- Modern chatbots use deep learning and transformers instead of simple rule-based approaches.
- Transformers use self-attention to understand context and relationships in language.
- Models like GPT, DialoGPT, and ChatGPT generate human-like responses.
- The pipeline involves intent detection, context handling, and response generation.
- Used in customer support, healthcare, banking, and virtual assistants.

### Code:

```
import torch

from transformers import AutoModelForCausalLM, AutoTokenizer

# Load pretrained DialoGPT model and tokenizer
tokenizer =
AutoTokenizer.from_pretrained("microsoft/DialoGPT-medium")

model =
AutoModelForCausalLM.from_pretrained("microsoft/DialoGPT-
medium")

def chat():
    chat_history_ids = None

    print("Chatbot: Hi! I'm a chatbot. Type 'quit' to
    exit.")

    while True:
        user_input = input("User: ")
        if user_input.lower() == 'quit':
```



```
        print("Chatbot: Goodbye!")
        break

# Encode user input
new_input_ids = tokenizer.encode(
    user_input + tokenizer.eos_token,
    return_tensors='pt')

# Append new user input to history
if chat_history_ids is not None:
    bot_input_ids = torch.cat([chat_history_ids,
new_input_ids], dim=-1)
else:
    bot_input_ids = new_input_ids
# ---- FIX: Add proper attention mask ----
attention_mask = torch.ones(bot_input_ids.shape,
dtype=torch.long)

# Generate response
chat_history_ids = model.generate(
    bot_input_ids,
    attention_mask=attention_mask,
    max_length=1000,
    pad_token_id=tokenizer.eos_token_id,
    temperature=0.9,
    do_sample=True,
    top_k=50,
    top_p=0.95 )

# Decode response
bot_response = tokenizer.decode(
    chat_history_ids[:, bot_input_ids.shape[-
1]:][0],
```

```
        skip_special_tokens=True    )

    print(f"Chatbot: {bot_response}")

if __name__ == "__main__":
    chat()
```

**Outputs:**

```
Chatbot: Hi! I'm a chatbot. Type 'quit' to exit.
User: hello how are you
Chatbot: good thanks
User: everything well
Chatbot: good and thanks
User: you're welcome
Chatbot: anytime man
User: yes but im a girl
Chatbot: you're welcome
User: ok bye then
Chatbot: yeah bye
User: quit
Chatbot: Goodbye!
```

## Practical No.: 04

**Aim:** To develop a recommendation system using collaborative filtering or deep learning approaches.

**Requirements:** pip install pandas numpy scikit-learn ipywidgets

### Theory:

- Recommender systems suggest items based on user preferences and behavior.
- Collaborative filtering uses similarities among users or items to generate recommendations.
- Deep learning methods (NCF, autoencoders, GNNs) detect hidden preference patterns.
- Hybrid systems combine collaborative and content-based filtering for better accuracy.
- Used by Netflix, Amazon, YouTube, and Spotify for personalization.

### Code:

```
# MOVIE RECOMMENDER - COLAB

# ----- STEP 1 : UPLOAD FILES -----
#uncomment this
# from google.colab import files
# import pandas as pd
# print("Upload movies.csv")
# movies_file = files.upload()
# print("Upload ratings.csv")
# ratings_file = files.upload()
# movies = pd.read_csv("movies.csv")
# ratings = pd.read_csv("ratings.csv")
# print("Movies Loaded:", movies.shape)
# print("Ratings Loaded:", ratings.shape)
```

```
# ----- STEP 2 : CLEAN TITLES -----  
import re  
def clean_title(title):  
    return re.sub("[^a-zA-Z0-9 ]", "", title)  
  
movies["clean_title"] = movies["title"].apply(clean_title)  
  
# ----- STEP 3 : BUILD TF-IDF SEARCH -----  
from sklearn.feature_extraction.text import TfidfVectorizer  
from sklearn.metrics.pairwise import cosine_similarity  
  
vectorizer = TfidfVectorizer(ngram_range=(1,2))  
tfidf = vectorizer.fit_transform(movies["clean_title"])  
  
def search_movie(title):  
    title = clean_title(title)  
    query_vec = vectorizer.transform([title])  
    similarity = cosine_similarity(query_vec,  
tfidf).flatten()  
    idx = similarity.argsort()[-5:][::-1]  
    return movies.iloc[idx][["id", "title"]] # Use 'id'  
instead of 'movieId'  
  
# ----- STEP 4 : COLLABORATIVE FILTERING -----  
import numpy as np  
  
def recommend(movie_id):  
  
    # Users who liked this movie  
    similar_users = ratings[  
        (ratings["movieId"] == movie_id) &  
        (ratings["rating"] >= 4)  
    ][["userId"].unique()]
```

```
if len(similar_users) == 0:
    return pd.DataFrame({"Message": ["Not enough similar
users found."]})

# Movies liked by similar users
similar_user_recs = ratings[
    (ratings["userId"].isin(similar_users)) &
    (ratings["rating"] >= 4)
][["movieId"]]

similar_user_recs = similar_user_recs.value_counts() /
len(similar_users)
similar_user_recs = similar_user_recs[similar_user_recs
> 0.10]

# Movies liked by everyone
all_users = ratings[
    (ratings["movieId"].isin(similar_user_recs.index)) &
    (ratings["rating"] >= 4)
]

all_user_recs = all_users["movieId"].value_counts() /
all_users["userId"].nunique()

# Calculate score
df = pd.concat([similar_user_recs, all_user_recs],
axis=1)
df.columns = ["similar", "all"]
df["score"] = df["similar"] / df["all"]
df = df.sort_values(by="score", ascending=False)

df = df.merge(movies, left_index=True, right_on="id") #
Join using 'id' instead of 'movieId'
return df[["score", "title", "genres"]].head(10)
```

```
# ----- STEP 5 : UI WIDGETS -----  
import ipywidgets as widgets  
from IPython.display import display, clear_output  
  
movie_input = widgets.Text(  
    value='Toy Story',  
    description='Search:',  
    placeholder='Type movie name...',  
)  
output_box = widgets.Output()  
  
def on_type(change):  
    with output_box:  
        clear_output()  
        text = change["new"]  
  
        if len(text) < 3:  
            print("Type at least 3 characters...")  
            return  
  
        print("Searching...")  
        results = search_movie(text)  
        display(results[["id", "title"]]) # Display 'id'  
and 'title' correctly  
  
        if results.empty:  
            print("No movies found.")  
            return  
  
        movie_id = int(results.iloc[0]["id"]) # Use 'id'  
instead of 'movieId'  
        print("\nRecommended Movies:")
```

```
display(recommend(movie_id)) # Pass 'id' to
recommend function
```

```
movie_input.observe(on_type, names="value")
display(movie_input, output_box)
```

## Outputs:

\*\*\* Search:

Searching...

|      | id     | title                          |
|------|--------|--------------------------------|
| 25   | 597    | Titanic                        |
| 1269 | 24575  | Raise the Titanic              |
| 3103 | 115872 | The Chambermaid on the Titanic |
| 1598 | 16871  | Drag Me to Hell                |
| 1599 | 293863 | The Age of Adaline             |

Recommended Movies:

|      | score    | title                 | genres                              |
|------|----------|-----------------------|-------------------------------------|
| 25   | 9.741935 | Titanic               | Drama Romance Thriller              |
| 583  | 5.506311 | Big Fish              | Adventure Fantasy Drama             |
| 3728 | 5.157495 | Whale Rider           | Drama Family                        |
| 3273 | 4.870968 | 8 Women               | Comedy Thriller Music Crime Mystery |
| 1366 | 4.870968 | The Devil Wears Prada | Comedy Drama Romance                |
| 2029 | 4.870968 | Underworld            | Fantasy Action Thriller             |
| 3446 | 4.639017 | Fahrenheit 9/11       | Documentary                         |
| 2912 | 4.531133 | Star Wars             | Adventure Action Science Fiction    |
| 2170 | 4.465054 | Psycho                | Drama Horror Thriller               |
| 568  | 4.329749 | xXx                   | Action Adventure Thriller           |

## Practical No.: 05

**Aim:** To Implement a Computer Vision project, such as object detection or image segmentation.

**Theory:**

- Computer vision extracts meaningful information from images and videos.
- Object detection identifies multiple objects and draws bounding boxes using YOLO, SSD, etc.
- Image segmentation classifies every pixel and is used in medical and autonomous systems.
- CNN-based encoder–decoder models like U-Net perform segmentation tasks effectively.
- Applications include face recognition, traffic monitoring, and robotics.

**Requirements:** pip install ultralytics opencv-python numpy matplotlib (in colab)

**Image:**



**Code:**

```
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
```



```
# Models
det_model = YOLO("yolov8n.pt")
seg_model = YOLO("yolov8n-seg.pt")

img = cv2.imread("streets.jpg")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Detection
det_res = det_model(img_rgb)[0].plot()

# Segmentation
seg_res = seg_model(img_rgb)[0].plot()

plt.figure(figsize=(10, 8))
plt.title("Object Detection")
plt.imshow(det_res)
plt.axis("off")
plt.show()

plt.figure(figsize=(10, 8))
plt.title("Image Segmentation")
plt.imshow(seg_res)
plt.axis("off")
plt.show()
```

**Output:**

```
0: 640x448 1 person, 4 cars, 1 truck, 1 backpack, 166.7ms  
Speed: 5.2ms preprocess, 166.7ms inference, 2.1ms postprocess per image at shape (1, 3, 640, 448)  
  
0: 640x448 1 person, 6 cars, 1 truck, 1 handbag, 241.6ms  
Speed: 4.4ms preprocess, 241.6ms inference, 21.2ms postprocess per image at shape (1, 3, 640, 448)
```

**Object Detection****Image Segmentation**

Saved as:  
- object\_detection\_result.jpg  
- segmentation\_result.jpg

## Practical No.: 06

**Aim:** To Train a Generative Adversarial Network (GAN) for generating realistic images.

Performed in Google Colab

**Requirements:** pip install torch torchvision matplotlib

### Theory:

- GANs contain two networks: Generator (creates images) and Discriminator (detects real vs fake).
- Both networks train in a competitive “minimax game” setup.
- Generator improves by learning to fool the discriminator over time.
- Techniques like BatchNorm, LeakyReLU, and Tanh help stable training.
- GANs are used for image synthesis, style transfer, super-resolution, and creative artwork.

### Code & Outputs:

#### Step 1- Import Libraries & Load the Dataset (MNIST Images)

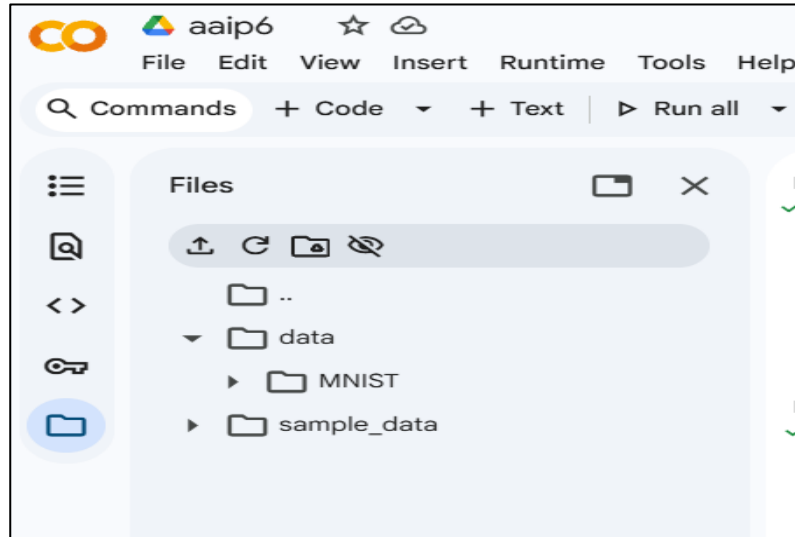
```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)) # Scale to [-1, 1]
])

dataset = torchvision.datasets.MNIST(root='./data',
train=True, transform=transform, download=True)

dataloader = torch.utils.data.DataLoader(dataset,
batch_size=128, shuffle=True)
```

|      |            |                                     |
|------|------------|-------------------------------------|
| 100% | ██████████ | 9.91M/9.91M [00:00<00:00, 17.8MB/s] |
| 100% | ██████████ | 28.9k/28.9k [00:00<00:00, 486kB/s]  |
| 100% | ██████████ | 1.65M/1.65M [00:00<00:00, 4.60MB/s] |
| 100% | ██████████ | 4.54k/4.54k [00:00<00:00, 5.15MB/s] |



## Step 2 - Define the Generator Network

*The Generator converts random noise → fake images.*

```
import torch.nn as nn

class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.main = nn.Sequential(
            nn.Linear(nz, ngf * 7 * 7),
            nn.BatchNorm1d(ngf * 7 * 7),
            nn.ReLU(True),
            nn.Unflatten(1, (ngf, 7, 7)),
            nn.ConvTranspose2d(ngf, ngf // 2, 4, 2, 1),
            nn.BatchNorm2d(ngf // 2),
            nn.ReLU(True),
            nn.ConvTranspose2d(ngf // 2, 1, 4, 2, 1),
            nn.Tanh())

    def forward(self, x):
        return self.main(x)
```

**Step 3 - Define the Discriminator Network**

*Discriminator checks whether an image is real or fake.*

```
class Discriminator(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.main = nn.Sequential(  
            nn.Conv2d(1, ndf, 4, 2, 1),  
            nn.LeakyReLU(0.2, inplace=True),  
            nn.Conv2d(ndf, ndf * 2, 4, 2, 1),  
            nn.BatchNorm2d(ndf * 2),  
            nn.LeakyReLU(0.2, inplace=True),  
            nn.Flatten(),  
            nn.Linear(ndf * 2 * 7 * 7, 1),  
            nn.Sigmoid())  
    def forward(self, x):  
        return self.main(x)
```

**Step 4 - Initialize Networks & Optimizers**

```
batch_size = 128  
image_size = 28  
nz = 100          # latent vector size  
ngf = 64          # generator feature maps  
ndf = 64          # discriminator feature maps  
epochs = 10  
  
import torch  
device = torch.device("cuda" if torch.cuda.is_available()  
else "cpu")  
print("Using:", device)  
  
G = Generator().to(device)  
D = Discriminator().to(device)
```

```
criterion = nn.BCELoss()

optimizerD = torch.optim.Adam(D.parameters(), lr=0.0002,
betas=(0.5, 0.999))

optimizerG = torch.optim.Adam(G.parameters(), lr=0.0002,
betas=(0.5, 0.999))
```

Using: cuda

### Step 5 - Training the GAN

*GAN training consists of 2 parts:*

- 1. Train Discriminator → Real vs Fake*
- 2. Train Generator → Fool the Discriminator*

```
for epoch in range(epochs):
    for i, (real, _) in enumerate(dataloader):
        real = real.to(device)
        b = real.size(0)
        # Create labels
        real_labels = torch.ones(b, 1, device=device)
        fake_labels = torch.zeros(b, 1, device=device)
        # Train Discriminator
        optimizerD.zero_grad()
        output_real = D(real)
        loss_real = criterion(output_real, real_labels)
        noise = torch.randn(b, nz, device=device)
        fake = G(noise)
        output_fake = D(fake.detach())
        loss_fake = criterion(output_fake, fake_labels)
        lossD = loss_real + loss_fake
        lossD.backward()
        optimizerD.step()
```

```

# Train Generator
optimizerG.zero_grad()
output = D(fake)
lossG = criterion(output, real_labels)
lossG.backward()
optimizerG.step()

print(f"Epoch [{epoch+1}/{epochs}]   LossD:
{lossD.item():.4f}   LossG: {lossG.item():.4f}")

```

|               |               |               |
|---------------|---------------|---------------|
| Epoch [1/10]  | LossD: 0.1828 | LossG: 2.4716 |
| Epoch [2/10]  | LossD: 0.1680 | LossG: 2.9420 |
| Epoch [3/10]  | LossD: 0.1837 | LossG: 3.1234 |
| Epoch [4/10]  | LossD: 0.2444 | LossG: 2.6126 |
| Epoch [5/10]  | LossD: 0.1759 | LossG: 3.0939 |
| Epoch [6/10]  | LossD: 0.2833 | LossG: 2.9363 |
| Epoch [7/10]  | LossD: 0.3859 | LossG: 3.8130 |
| Epoch [8/10]  | LossD: 0.2449 | LossG: 3.7634 |
| Epoch [9/10]  | LossD: 0.3328 | LossG: 2.5911 |
| Epoch [10/10] | LossD: 0.2007 | LossG: 2.9993 |

### Step 6 - Generate Images After Training

```

import torchvision.utils as vutils
noise = torch.randn(16, nz, device=device)
fake = G(noise).cpu().detach()
# Convert from [-1,1] to [0,1]
fake = (fake + 1) / 2
grid = vutils.make_grid(fake, nrow=4)
plt.figure(figsize=(6,6))
plt.imshow(grid.permute(1, 2, 0))
plt.axis("off")
plt.show()

```

### Final Output:



## Practical No.: 07

**Aim:** To Apply reinforcement learning algorithms to solve complex decision-making problems.

**Requirements:** pip install torch numpy matplotlib pygame

Model.py = pip install torch numpy

Helper.py = pip install matplotlib ipython

Game.py = pip install pygame numpy

### Theory:

- RL trains an agent to interact with an environment and learn through rewards and penalties.
- The agent observes a state, takes an action, and receives feedback to learn an optimal policy.
- Popular RL algorithms include Q-Learning, DQN, SARSA, PPO, and Actor-Critic.
- Deep RL uses neural networks to handle complex environments.
- Applications include robotics, games (AlphaGo), finance, and autonomous driving.

### Code:

#### aaip7.py

```
# train.py
import torch
import random
import numpy as np
from collections import deque
from game import SnakeGameAI, Direction, Point
from model import Linear_QNet, QTrainer
from helper import plot
MAX_MEMORY = 100_000
BATCH_SIZE = 1000
LR = 0.001

print('Reinforcement Learning (RL) - Deep Q-Learning (DQN)')
class Agent:
    def __init__(self):
        self.n_games = 0
        self.epsilon = 0 # randomness
        self.gamma = 0.9 # discount rate
        self.memory = deque(maxlen=MAX_MEMORY)
        self.model = Linear_QNet(11, 256, 3)
        self.trainer = QTrainer(self.model, lr=LR,
gamma=self.gamma)
```



```
def get_state(self, game):
    head = game.snake[0]
    point_l = Point(head.x - 20, head.y)
    point_r = Point(head.x + 20, head.y)
    point_u = Point(head.x, head.y - 20)
    point_d = Point(head.x, head.y + 20)

    dir_l = game.direction == Direction.LEFT
    dir_r = game.direction == Direction.RIGHT
    dir_u = game.direction == Direction.UP
    dir_d = game.direction == Direction.DOWN
    state = [
        # Danger straight
        (dir_r and game.is_collision(point_r)) or
        (dir_l and game.is_collision(point_l)) or
        (dir_u and game.is_collision(point_u)) or
        (dir_d and game.is_collision(point_d)),

        # Danger right
        (dir_u and game.is_collision(point_r)) or
        (dir_d and game.is_collision(point_l)) or
        (dir_l and game.is_collision(point_u)) or
        (dir_r and game.is_collision(point_d)),

        # Danger left
        (dir_d and game.is_collision(point_r)) or
        (dir_u and game.is_collision(point_l)) or
        (dir_r and game.is_collision(point_u)) or
        (dir_l and game.is_collision(point_d)),

        # Move direction
        dir_l,
        dir_r,
        dir_u,
        dir_d,

        # Food location
        game.food.x < head.x, # food left
        game.food.x > head.x, # food right
        game.food.y < head.y, # food up
        game.food.y > head.y, # food down
    ]
    return np.array(state, dtype=int)

def remember(self, state, action, reward, next_state,
done):
    self.memory.append((state, action, reward,
next_state, done))

def train_long_memory(self):
    if len(self.memory) > BATCH_SIZE:
```

```
        mini_sample = random.sample(self.memory,
BATCH_SIZE)
    else:
        mini_sample = self.memory
        states, actions, rewards, next_states, dones =
zip(*mini_sample)
        self.trainer.train_step(states, actions, rewards,
next_states, dones)

    def train_short_memory(self, state, action, reward,
next_state, done):
        self.trainer.train_step(state, action, reward,
next_state, done)

    def get_action(self, state):
        # Exploration vs Exploitation
        self.epsilon = 80 - self.n_games
        final_move = [0, 0, 0]

        if random.randint(0, 200) < self.epsilon:
            move = random.randint(0, 2)
            final_move[move] = 1
        else:
            state0 = torch.tensor(state, dtype=torch.float)
            prediction = self.model(state0)
            move = torch.argmax(prediction).item()
            final_move[move] = 1
        return final_move

    def train():
        plot_scores = []
        plot_mean_scores = []
        total_score = 0
        record = 0
        agent = Agent()
        game = SnakeGameAI()

        while True:
            #print("Loop running...") # 🐞 Debug line to check
            loop execution

            # Get old state
            state_old = agent.get_state(game)

            # Get move
            final_move = agent.get_action(state_old)

            # Perform move and get new state
            reward, done, score = game.play_step(final_move)
            state_new = agent.get_state(game)
```

```
# Train short memory
agent.train_short_memory(state_old, final_move,
reward, state_new, done)

# Remember in memory
agent.remember(state_old, final_move, reward,
state_new, done)

if done:
    # Train long memory
    game.reset()
    agent.n_games += 1
    agent.train_long_memory()

    if score > record:
        record = score
        agent.model.save()

    print('Game', agent.n_games, 'Score', score,
'Record:', record)

    plot_scores.append(score)
    total_score += score
    mean_score = total_score / agent.n_games
    plot_mean_scores.append(mean_score)
    plot(plot_scores, plot_mean_scores)

if __name__ == '__main__':
    train()
```

### **model.py**

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import numpy as np
import os

class Linear_QNet(nn.Module):
    def __init__(self, input_size, hidden_size,
output_size):
        super().__init__()
        self.linear1 = nn.Linear(input_size, hidden_size)
        self.linear2 = nn.Linear(hidden_size, output_size)
    def forward(self, x):
        x = F.relu(self.linear1(x))
        x = self.linear2(x)
        return x
```

```
def save(self, file_name='model.pth'):
    model_folder_path = './model'
    if not os.path.exists(model_folder_path):
        os.makedirs(model_folder_path)
    file_name = os.path.join(model_folder_path,
file_name)
    torch.save(self.state_dict(), file_name)

class QTrainer:
    def __init__(self, model, lr, gamma):
        self.lr = lr
        self.gamma = gamma
        self.model = model
        self.optimizer = optim.Adam(model.parameters()),
lr=self.lr)
        self.criterion = nn.MSELoss()

    def train_step(self, state, action, reward, next_state,
done):
        state = torch.tensor(np.array(state),
dtype=torch.float)
        next_state = torch.tensor(np.array(next_state),
dtype=torch.float)
        action = torch.tensor(np.array(action),
dtype=torch.long)
        reward = torch.tensor(np.array(reward),
dtype=torch.float)

        if len(state.shape) == 1:
            state = torch.unsqueeze(state, 0)
            next_state = torch.unsqueeze(next_state, 0)
            action = torch.unsqueeze(action, 0)
            reward = torch.unsqueeze(reward, 0)
            done = (done,)

        pred = self.model(state)
        target = pred.clone()

        for idx in range(len(done)):
            Q_new = reward[idx]
            if not done[idx]:
                Q_new = reward[idx] + self.gamma *
torch.max(self.model(next_state[idx]))
            target[idx][torch.argmax(action[idx]).item()] =
Q_new

        self.optimizer.zero_grad()
        loss = self.criterion(target, pred)
```

```
loss.backward()  
self.optimizer.step()
```

### **helper.py**

```
import matplotlib.pyplot as plt  
from IPython import display  
plt.ion()  
  
def plot(scores, mean_scores):  
    display.clear_output(wait=True)  
    #display.display(plt.gcf())  
    plt.clf()  
    plt.title('Training...')  
    plt.xlabel('Number of Games')  
    plt.ylabel('Score')  
    plt.plot(scores)  
    plt.plot(mean_scores)  
    plt.ylim(ymin=0)  
    plt.text(len(scores)-1, scores[-1], str(scores[-1]))  
    plt.text(len(mean_scores)-1, mean_scores[-1],  
str(round(mean_scores[-1], 2)))  
    plt.show(block=False)  
    plt.pause(0.1)
```

### **game.py**

```
import pygame  
import random  
from enum import Enum  
from collections import namedtuple  
import numpy as np  
pygame.init()  
font = pygame.font.Font(None, 25)  
  
class Direction(Enum):  
    RIGHT = 2  
    LEFT = 1  
    UP = 6  
    DOWN = 3  
Point = namedtuple('Point', 'x, y')  
  
BLOCK_SIZE = 10  
SPEED = 10 # You can lower this (e.g. 20) if too fast  
  
WHITE = (255, 255, 255)  
RED = (200, 0, 0)  
BLUE1 = (0, 0, 255)  
BLUE2 = (0, 100, 255)  
BLACK = (0, 0, 0)
```

```
class SnakeGameAI:
    def __init__(self, w=640, h=480):
        self.w = w
        self.h = h
        self.display = pygame.display.set_mode((self.w,
self.h))
        pygame.display.set_caption('Snake RL DQN')
        self.clock = pygame.time.Clock()
        self.BLOCK_SIZE = 20
        self.reset()

    def reset(self):
        self.direction = Direction.RIGHT
        self.head = Point(self.w / 2, self.h / 2)
        self.snake = [self.head,
                        Point(self.head.x - BLOCK_SIZE,
self.head.y),
                        Point(self.head.x - (2 * BLOCK_SIZE),
self.head.y)]
        self.score = 0
        self.food = None
        self._place_food()
        self.frame_iteration = 0

    def _place_food(self):
        x = random.randint(0, (self.w - BLOCK_SIZE) //
BLOCK_SIZE) * BLOCK_SIZE
        y = random.randint(0, (self.h - BLOCK_SIZE) //
BLOCK_SIZE) * BLOCK_SIZE
        self.food = Point(x, y)
        if self.food in self.snake:
            self._place_food()

    def play_step(self, action):
        self.frame_iteration += 1
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()

        # action -> [straight, right, left]
        self._move(action)
        self.snake.insert(0, self.head)

        reward = 0
        game_over = False
        if self.is_collision() or self.frame_iteration > 100
* len(self.snake):
            game_over = True
            reward = -10
            return reward, game_over, self.score
```

```

        if self.head == self.food:
            self.score += 1
            reward = 10
            self._place_food()
        else:
            self.snake.pop()
            self._update_ui()
            self.clock.tick(SPEED)
            return reward, game_over, self.score

    def is_collision(self, pt=None):
        if pt is None:
            pt = self.head
        # hits boundary
        if pt.x > self.w - BLOCK_SIZE or pt.x < 0 or pt.y >
self.h - BLOCK_SIZE or pt.y < 0:
            return True
        # hits itself
        if pt in self.snake[1:]:
            return True
        return False

    def _update_ui(self):
        self.display.fill(BLACK)
        for i, pt in enumerate(self.snake):
            color_factor = i / len(self.snake)
            color = (
                int(0 + 100 * color_factor), # Red channel
                int(0 + 100 * (1 - color_factor)), # Green
channel (inverse effect)
                255 # Blue channel (constant)
            )
            pygame.draw.rect(self.display, color,
pygame.Rect(pt.x, pt.y, BLOCK_SIZE, BLOCK_SIZE))

        # Draw the food
        pygame.draw.rect(self.display, RED,
pygame.Rect(self.food.x, self.food.y, BLOCK_SIZE,
BLOCK_SIZE))

        # Display the score
        text = font.render("Score: " + str(self.score),
True, WHITE)
        self.display.blit(text, [0, 0])
        pygame.display.flip()

    def _move(self, action):
        # [straight, right, left]
        clock_wise = [Direction.RIGHT, Direction.DOWN,
Direction.LEFT, Direction.UP]

```

```

idx = clock_wise.index(self.direction)

if np.array_equal(action, [1, 0, 0]):
    new_dir = clock_wise[idx] # no change
elif np.array_equal(action, [0, 1, 0]):
    next_idx = (idx + 1) % 4
    new_dir = clock_wise[next_idx] # right turn
else: # [0, 0, 1]
    next_idx = (idx - 1) % 4
    new_dir = clock_wise[next_idx] # left turn

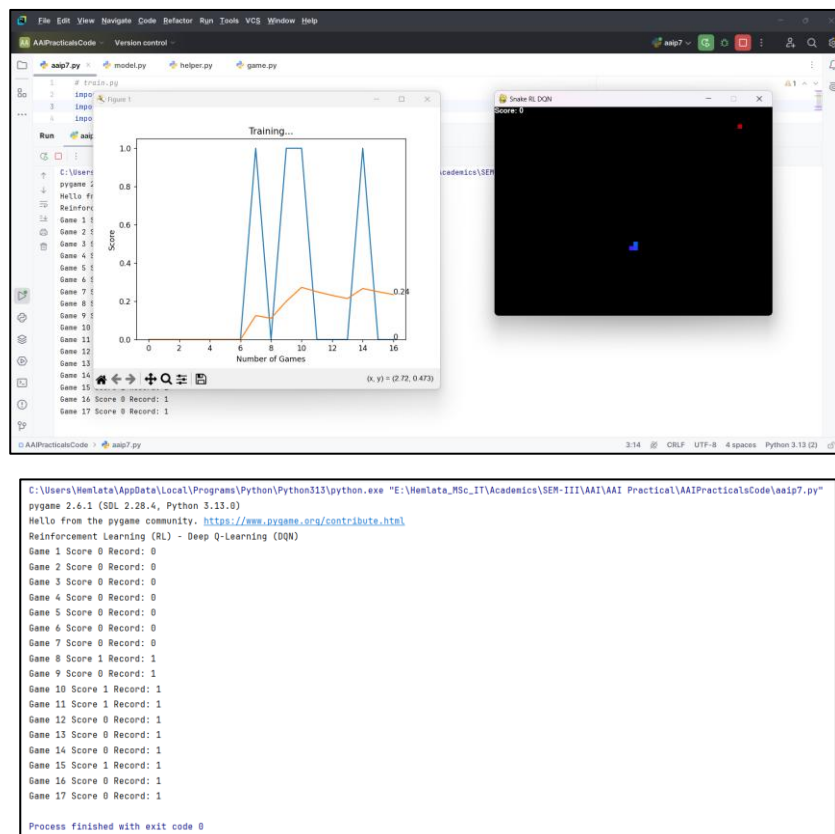
self.direction = new_dir

x = self.head.x
y = self.head.y
if self.direction == Direction.RIGHT:
    x += BLOCK_SIZE
elif self.direction == Direction.LEFT:
    x -= BLOCK_SIZE
elif self.direction == Direction.DOWN:
    y += BLOCK_SIZE
elif self.direction == Direction.UP:
    y -= BLOCK_SIZE

self.head = Point(x, y)

```

## Outputs:





## Practical No.: 08

**Aim:** To Utilize transfer learning to improve model performance on limited datasets.

### Theory:

- Transfer learning reuses a model trained on a large dataset for a new related task.
- Models like ResNet, VGG, and EfficientNet pretrained on ImageNet serve as feature extractors.
- Earlier layers are frozen while later layers are fine-tuned for the new dataset.
- Reduces computation, prevents overfitting, and improves accuracy on small datasets.
- Widely used in medical imaging, security, NLP tasks, and chatbots.

**Requirements:** pip install torch torchvision matplotlib

### Code:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim import lr_scheduler
import numpy as np
import torchvision
from torchvision import datasets, models, transforms
import matplotlib.pyplot as plt
import time
import os
import copy

# -----
# DEVICE SETUP
# -----
device = torch.device("cuda:0" if torch.cuda.is_available()
else "cpu")
print("🔥 Using device:", device)

# -----
# DATA TRANSFORMS
# -----
mean = np.array([0.5, 0.5, 0.5])
std = np.array([0.25, 0.25, 0.25])

data_transforms = {
    'train': transforms.Compose([
```

```

        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ]),
}

# -----
# LOAD DATASET
# -----
data_dir = 'data/hymenoptera_data'

image_datasets = {
    x: datasets.ImageFolder(os.path.join(data_dir, x),
        data_transforms[x])
    for x in ['train', 'val']
}

dataloaders = {
    x: torch.utils.data.DataLoader(image_datasets[x],
        batch_size=4, shuffle=True, num_workers=0)
    for x in ['train', 'val']
}

dataset_sizes = {x: len(image_datasets[x]) for x in
    image_datasets}
class_names = image_datasets['train'].classes
print("✂ Classes:", class_names)

# -----
# SHOW SAMPLE IMAGE
# -----
def imshow(inp, title):
    inp = inp.numpy().transpose((1, 2, 0))
    inp = std * inp + mean
    inp = np.clip(inp, 0, 1)
    plt.imshow(inp)
    plt.title(title)
    plt.show()

inputs, classes = next(iter(dataloaders["train"]))
out = torchvision.utils.make_grid(inputs)
imshow(out, title=[class_names[x] for x in classes])

# -----

```

```

# TRAINING FUNCTION
# -----
def train_model(model, criterion, optimizer, scheduler,
num_epochs=25):
    since = time.time()
    best_acc = 0.0
    best_weights = copy.deepcopy(model.state_dict())

    for epoch in range(num_epochs):
        print(f"\n📅 Epoch {epoch}/{num_epochs-1}")
        print("-" * 20)

        for phase in ["train", "val"]:
            model.train() if phase == "train" else
model.eval()

            running_loss = 0.0
            running_corrects = 0

            for inputs, labels in dataloaders[phase]:
                inputs = inputs.to(device)
                labels = labels.to(device)

                optimizer.zero_grad()

                with torch.set_grad_enabled(phase ==
"train"):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    if phase == "train":
                        loss.backward()
                        optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds ==
labels.data)

            if phase == "train":
                scheduler.step()

            epoch_loss = running_loss / dataset_sizes[phase]
            epoch_acc = running_corrects.double() /
dataset_sizes[phase]

            print(f"{phase} Loss: {epoch_loss:.4f} Acc:
{epoch_acc:.4f}")

            if phase == "val" and epoch_acc > best_acc:
                best_acc = epoch_acc

```

```

        best_weights =
copy.deepcopy(model.state_dict())

    time_elapsed = time.time() - since
    print(f"\n⌚ Training complete in
{time_elapsed//60:.0f}m {time_elapsed%60:.0f}s")
    print(f"🏆 Best Validation Accuracy: {best_acc:.4f}")

    model.load_state_dict(best_weights)
    return model

#
=====
=
# ❶ FINETUNE RESNET18 (TRAIN ALL LAYERS)
#
=====
=
print("\n🚀 Training FULL ResNet18")

model = models.resnet18(pretrained=True)
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, 2)    # 2 classes
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.001)
scheduler = lr_scheduler.StepLR(optimizer, step_size=7,
gamma=0.1)

model = train_model(model, criterion, optimizer, scheduler,
num_epochs=10)

#
=====
=
# ❷ CONVNET AS FIXED FEATURE EXTRACTOR
#
=====
=
print("\n🧊 FREEZING ALL LAYERS EXCEPT LAST")

model_conv = models.resnet18(pretrained=True)

for param in model_conv.parameters():
    param.requires_grad = False

num_ftrs = model_conv.fc.in_features
model_conv.fc = nn.Linear(num_ftrs, 2)
model_conv = model_conv.to(device)

```

```

criterion = nn.CrossEntropyLoss()
optimizer_conv = optim.SGD(model_conv.fc.parameters(),
                             lr=0.001, momentum=0.9)
scheduler_conv = lr_scheduler.StepLR(optimizer_conv,
                                       step_size=7, gamma=0.1)

model_conv = train_model(model_conv, criterion,
                          optimizer_conv, scheduler_conv, num_epochs=10)

```

## Output:



```

@ Epoch 4/9
-----
train Loss: 0.4427 Acc: 0.7992
val Loss: 0.2456 Acc: 0.9216

@ Epoch 5/9
-----
train Loss: 0.4342 Acc: 0.8033
val Loss: 0.2384 Acc: 0.9150

@ Epoch 6/9
-----
train Loss: 0.3743 Acc: 0.8443
val Loss: 0.2152 Acc: 0.9150

@ Epoch 7/9
-----
train Loss: 0.3834 Acc: 0.8074
val Loss: 0.2158 Acc: 0.9216

@ Epoch 8/9
-----
train Loss: 0.3573 Acc: 0.8484
val Loss: 0.2454 Acc: 0.9085

@ Epoch 9/9
-----
train Loss: 0.3534 Acc: 0.8648
val Loss: 0.2096 Acc: 0.9150

🔧 Training complete in 6m 34s
🔧 Best Validation Accuracy: 0.9216

@ FREEZING ALL LAYERS EXCEPT LAST

@ Epoch 0/9
-----
train Loss: 0.5312 Acc: 0.7623
val Loss: 0.2801 Acc: 0.8889

```

```

@ Epoch 2/9
-----
train Loss: 0.5368 Acc: 0.7459
val Loss: 0.2295 Acc: 0.9150

@ Epoch 3/9
-----
train Loss: 0.3975 Acc: 0.8115
val Loss: 0.4083 Acc: 0.8235

@ Epoch 4/9
-----
train Loss: 0.4199 Acc: 0.8156
val Loss: 0.1830 Acc: 0.9412

@ Epoch 5/9
-----
train Loss: 0.4333 Acc: 0.8074
val Loss: 0.1829 Acc: 0.9477

@ Epoch 6/9
-----
train Loss: 0.6317 Acc: 0.7049
val Loss: 0.2994 Acc: 0.8954

@ Epoch 7/9
-----
train Loss: 0.4446 Acc: 0.8074
val Loss: 0.1863 Acc: 0.9346

@ Epoch 8/9
-----
train Loss: 0.2975 Acc: 0.8730
val Loss: 0.2120 Acc: 0.9477
-

@ Epoch 9/9
-----
train Loss: 0.3088 Acc: 0.8852
val Loss: 0.1955 Acc: 0.9542

🔧 Training complete in 4m 38s
🔧 Best Validation Accuracy: 0.9542

```

## Practical No.: 09

**Aim:** To Build a deep learning model for time series forecasting or anomaly detection.

### Theory:

- Time series forecasting predicts future values using past historical data.
- Deep models like LSTM, GRU, and Temporal CNNs capture long-term patterns.
- Anomaly detection identifies sudden deviations from expected patterns.
- Performance is evaluated using metrics like RMSE, MAE, and MAPE.
- Used in IoT monitoring, finance, weather prediction, and predictive maintenance.

**Requirements:** pip install pandas numpy matplotlib seaborn xgboost scikit-learn

### Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import xgboost as xgb
from sklearn.metrics import mean_squared_error

# PLOT SETTINGS
# -----
color_pal = sns.color_palette()
plt.style.use("fivethirtyeight")

# LOAD DATA
# -----
df = pd.read_csv("PJME_hourly.csv")
df = df.set_index("Datetime")
df.index = pd.to_datetime(df.index)
df = df.sort_index()
df.plot(style=".", figsize=(15, 5), title="PJME Energy Use
in MW", color=color_pal[0])
plt.show()

# TRAIN / TEST SPLIT
# -----
train = df.loc[df.index < "01-01-2015"]
test = df.loc[df.index >= "01-01-2015"]
fig, ax = plt.subplots(figsize=(15, 5))
train.plot(ax=ax, label="Training Set")
test.plot(ax=ax, label="Test Set")
```

```
ax.axvline("01-01-2015", color="black", ls="--")
plt.legend()
plt.title("Train/Test Split")
plt.show()

# VISUALIZE WEEK DATA SAMPLE
# -----
df.loc[(df.index > "01-01-2010") & (df.index < "01-08-
2010")] \
    .plot(figsize=(15, 5), title="Week Of Data")
plt.show()

# FEATURE ENGINEERING FUNCTION
# -----
def create_features(df):
    df = df.copy()
    df["hour"] = df.index.hour
    df["dayofweek"] = df.index.dayofweek
    df["quarter"] = df.index.quarter
    df["month"] = df.index.month
    df["year"] = df.index.year
    df["dayofyear"] = df.index.dayofyear
    return df

df = create_features(df)

# BOX PLOT ANALYSIS
# -----
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x="hour", y="PJME_MW", hue="hour",
palette="Blues", legend=False)
plt.title("MW by Hour")
plt.show()

plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x="month", y="PJME_MW", hue="month",
palette="Blues", legend=False)
plt.title("MW by Month")
plt.show()

# TRAIN MODEL
# -----
train = create_features(train)
test = create_features(test)

FEATURES = ["dayofyear", "hour", "dayofweek", "quarter",
"month", "year"]
TARGET = "PJME_MW"
X_train = train[FEATURES]
y_train = train[TARGET]
X_test = test[FEATURES]
```



```
y_test = test[TARGET]
reg = xgb.XGBRegressor(
    n_estimators=1000,
    early_stopping_rounds=50,
    objective="reg:squarederror",
    max_depth=3,
    learning_rate=0.01
)
reg.fit(
    X_train,
    y_train,
    eval_set=[(X_train, y_train), (X_test, y_test)],
    verbose=100
)

# FEATURE IMPORTANCE
# -----
fi = pd.DataFrame(
    data=reg.feature_importances_,
    index=reg.feature_names_in_,
    columns=["importance"]
)

fi.sort_values("importance").plot(kind="barh", figsize=(6,
4), title="Feature Importance")
plt.show()

# MAKE FORECAST
# -----
test["prediction"] = reg.predict(X_test)

df = df.merge(test[["prediction"]], left_index=True,
right_index=True, how="left")

ax = df[["PJME_MW"]].plot(figsize=(15, 5))
df["prediction"].plot(ax=ax, style=".")
plt.title("Prediction vs Real Value (Full Range)")
plt.show()

# ZOOM INTO ONE WEEK FORECAST
# -----
week_slice = df.loc["04-01-2018":"04-08-2018"]
week_slice[["PJME_MW"]].plot(figsize=(15, 5), title="True vs
Predicted - One Week")
week_slice["prediction"].plot(style=".")
plt.legend(["Truth Data", "Prediction"])
plt.show()

# RMSE SCORE
# -----
```

```

score = np.sqrt(mean_squared_error(test["PJME_MW"],
test["prediction"]))
print(f"RMSE Score on Test Set: {score:.2f}")

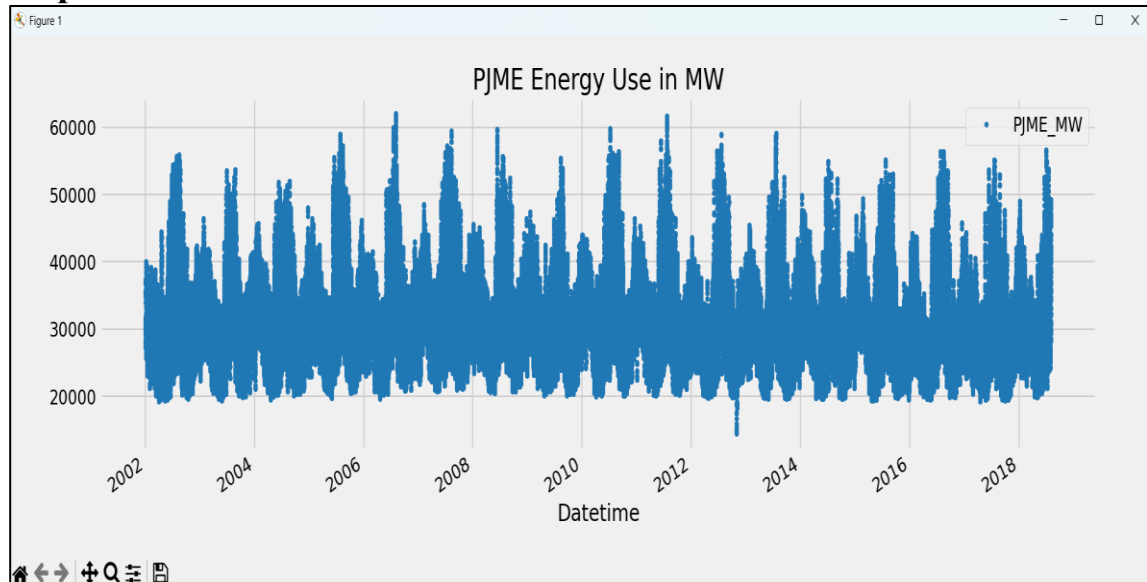
# BEST / WORST PREDICTED DAYS
# -----
test["error"] = np.abs(test[TARGET] - test["prediction"])
test["date"] = test.index.date

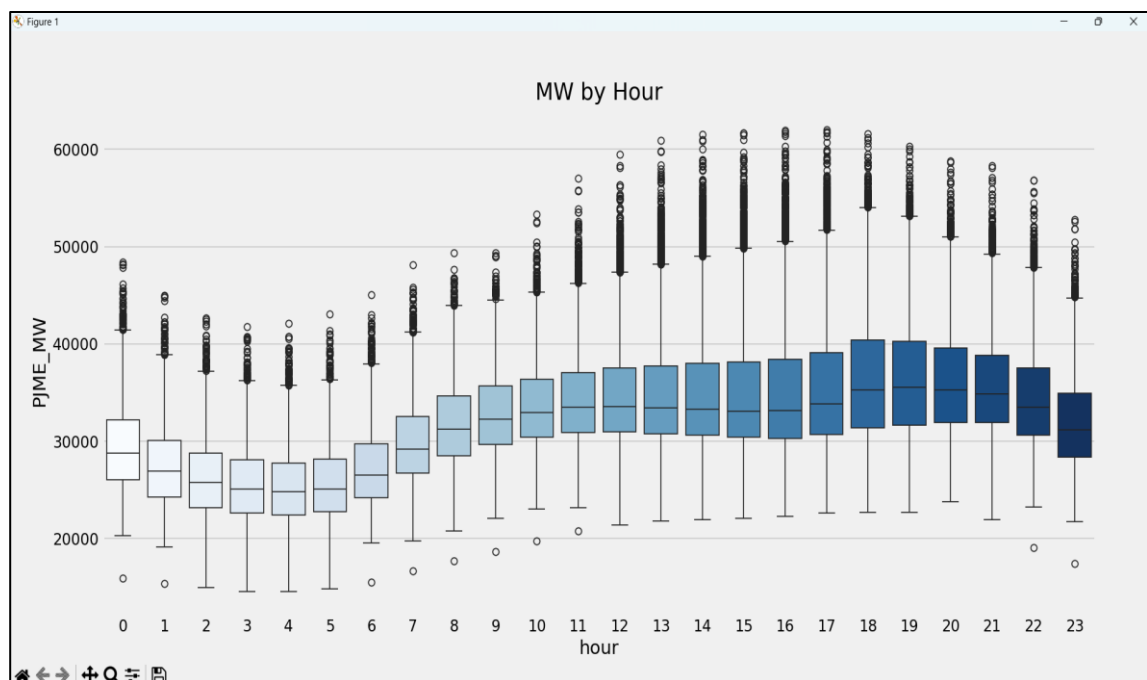
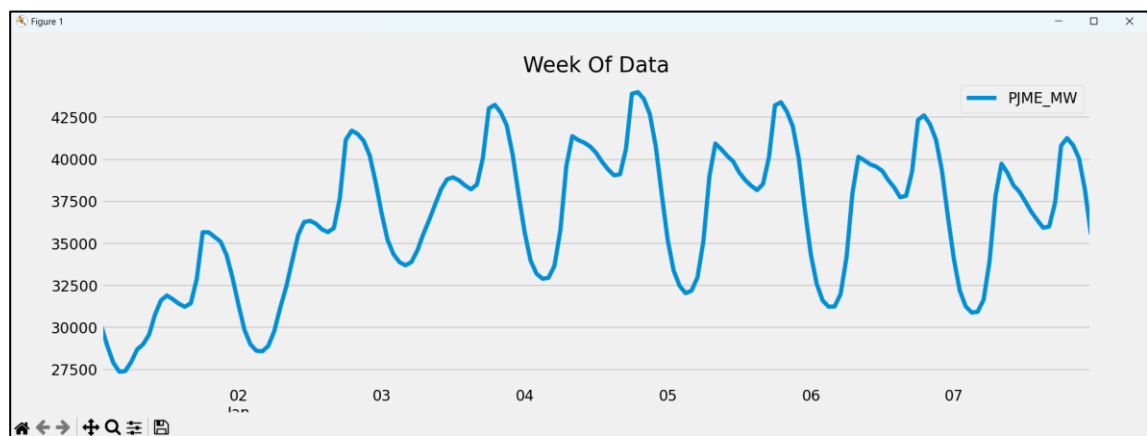
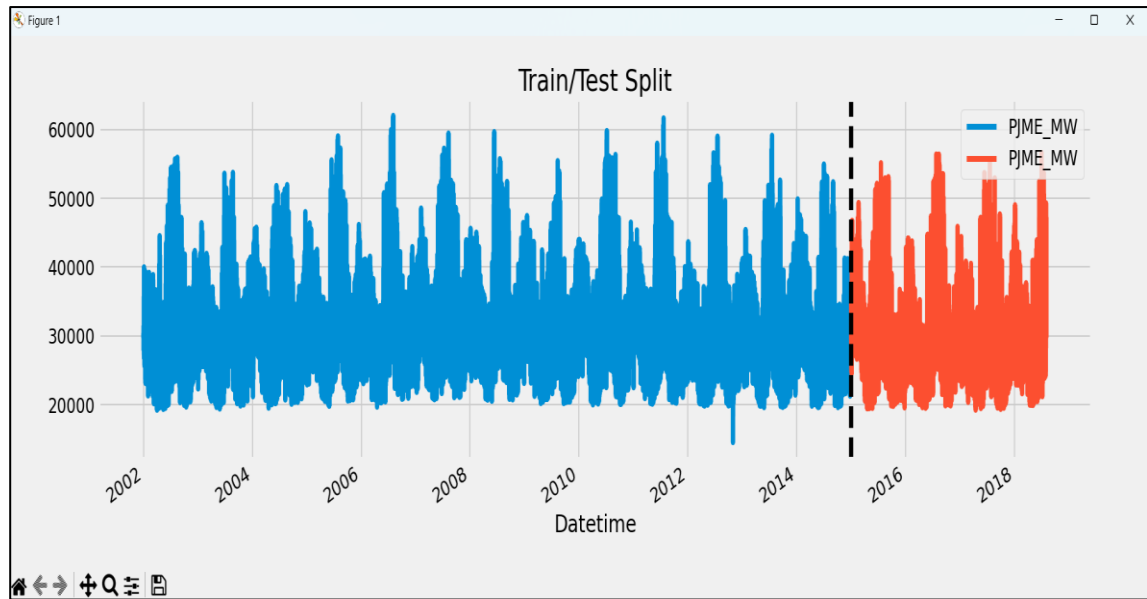
print("\n🌀 Worst 10 Days (Highest Prediction Error)")
print(test.groupby("date")["error"].mean().sort_values(ascending=False).head(10))

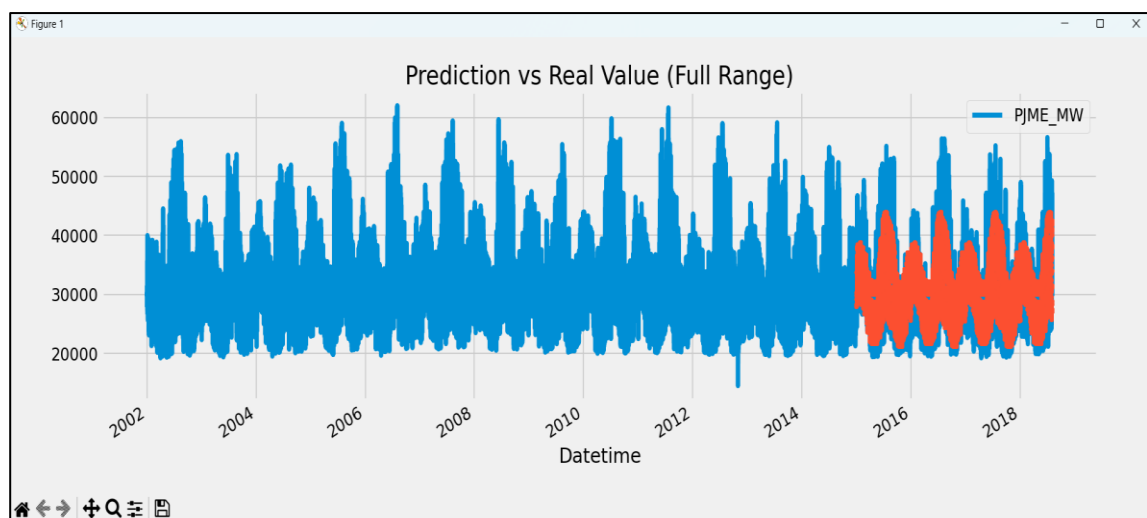
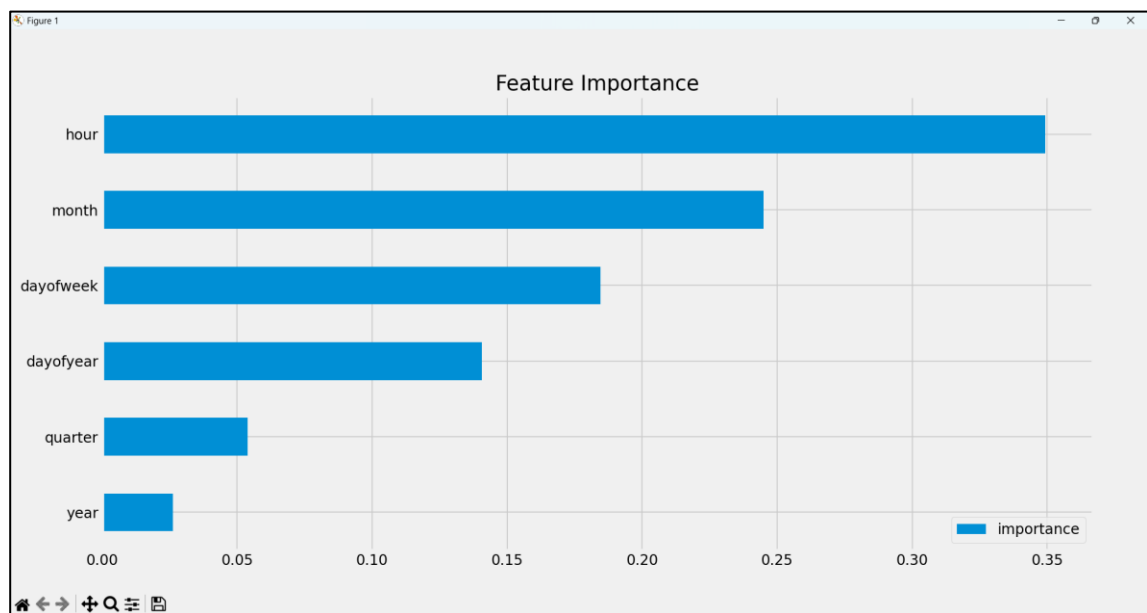
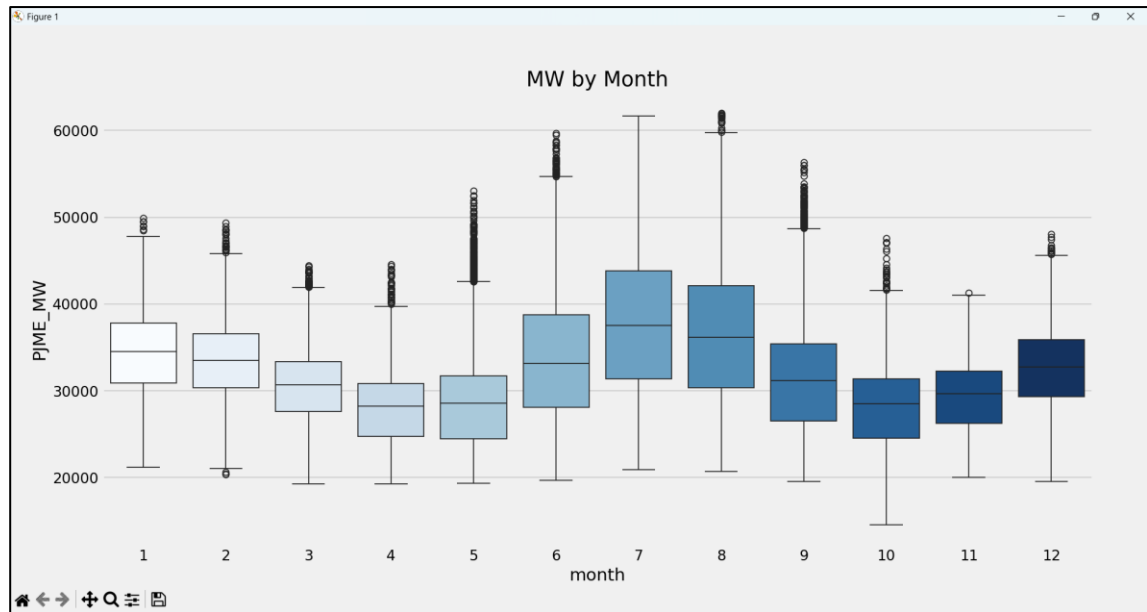
print("\n🌀 Best 10 Days (Lowest Prediction Error)")
print(test.groupby("date")["error"].mean().sort_values().head(10))

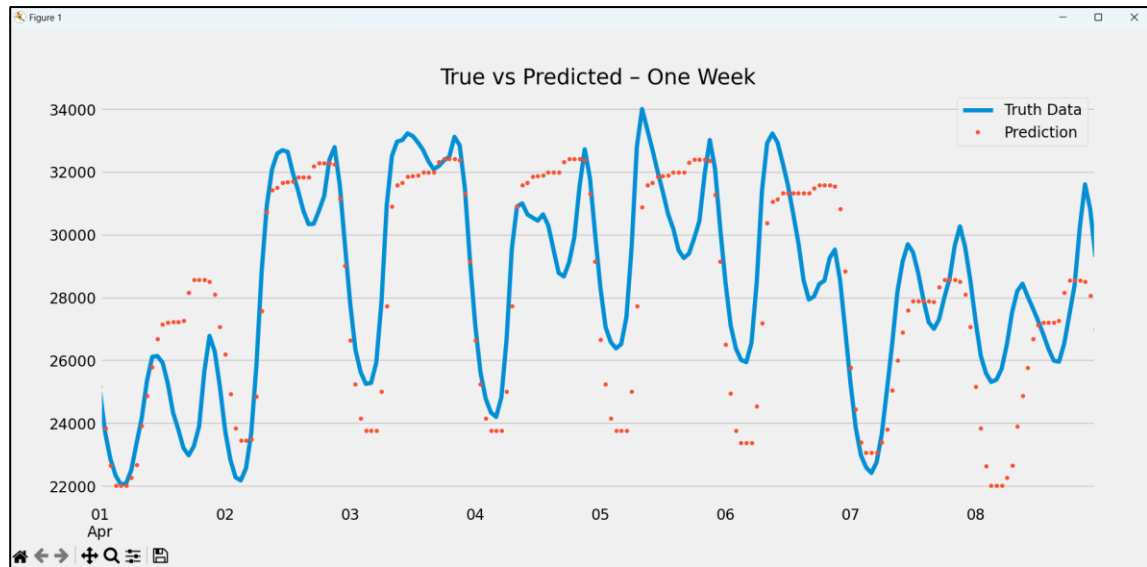
```

### Output:









```

[0] validation_0-rmse:6424.52298 validation_1-rmse:6496.44816
[100] validation_0-rmse:4623.48045 validation_1-rmse:4903.65897
[200] validation_0-rmse:3965.12971 validation_1-rmse:4359.30685
[300] validation_0-rmse:3594.48713 validation_1-rmse:4065.48650
[400] validation_0-rmse:3400.15690 validation_1-rmse:3919.24289
[500] validation_0-rmse:3281.93774 validation_1-rmse:3838.02851
[600] validation_0-rmse:3204.63821 validation_1-rmse:3786.28747
[700] validation_0-rmse:3151.88956 validation_1-rmse:3758.18082
[800] validation_0-rmse:3113.08737 validation_1-rmse:3738.25606
[900] validation_0-rmse:3083.34772 validation_1-rmse:3729.31072
[974] validation_0-rmse:3062.81046 validation_1-rmse:3727.56716
RMSE Score on Test Set: 3727.15

```

● Worst 10 Days (Highest Prediction Error)

```

date
2016-08-13 12843.450195
2016-08-14 12730.175049
2015-02-20 11204.544596
2016-09-09 10940.650472
2016-09-10 10873.265625
2018-01-06 10626.123372
2016-08-12 10020.220052
2015-02-21 10013.332438
2015-02-16 9896.746012
2018-01-07 9829.796712
Name: error, dtype: float64

```

● Best 10 Days (Lowest Prediction Error)

```

date
2017-10-24 335.338379
2015-10-28 426.195964
2016-10-27 542.505534
2015-05-06 563.494385
2017-10-15 565.484212
2018-05-16 570.238851
2015-11-06 684.823568
2017-04-07 687.363770
2015-10-03 711.464844
2016-12-29 713.021077
Name: error, dtype: float64

```

## Practical No.: 10

**Aim:** To Implement a machine learning pipeline for automated feature engineering and model selection.

**Requirements:** !pip install mljar-supervised (in colab)

### Theory:

- ML pipelines automate preprocessing, feature engineering, training, and evaluation.
- AutoML frameworks like TPOT and EvalML automatically test multiple models.
- Pipelines include scaling, encoding, model selection, and hyperparameter optimization.
- Bayesian search improves tuning efficiency over grid or random search.
- Used widely in enterprise ML, MLOps, and automated decision systems.

### Code:

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split,
GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score,
precision_score, recall_score, roc_auc_score

# -----
# LOAD DATA
# -----
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)

# TRAIN/TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# -----
# DEFINE MODELS AND PARAM GRID
# -----
```

```
pipelines = {
    'logistic': Pipeline([
        ('scaler', StandardScaler()),
        ('clf', LogisticRegression(max_iter=1000))
    ]),
    'random_forest': Pipeline([
        ('scaler', StandardScaler()),
        ('clf', RandomForestClassifier())
    ]),
    'gradient_boosting': Pipeline([
        ('scaler', StandardScaler()),
        ('clf', GradientBoostingClassifier())
    ])
}

param_grids = {
    'logistic': {
        'clf__C': [0.01, 0.1, 1, 10]
    },
    'random_forest': {
        'clf__n_estimators': [50, 100, 200],
        'clf__max_depth': [3, 5, 10, None]
    },
    'gradient_boosting': {
        'clf__n_estimators': [50, 100],
        'clf__learning_rate': [0.01, 0.1, 0.2]
    }
}

# -----
# AUTOMATED MODEL SELECTION
# -----
best_model = None
best_score = 0
results = []

for name, pipeline in pipelines.items():
    grid = GridSearchCV(pipeline, param_grids[name], cv=5,
        scoring='accuracy')
    grid.fit(X_train, y_train)
    y_pred = grid.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    roc_auc = roc_auc_score(y_test,
        grid.predict_proba(X_test)[:,1])

    results.append({
        'model': name,
```



```

        'best_params': grid.best_params_,
        'accuracy': acc,
        'f1': f1,
        'precision': precision,
        'recall': recall,
        'roc_auc': roc_auc
    })

    if acc > best_score:
        best_score = acc
        best_model = grid.best_estimator_

# -----
# DISPLAY RESULTS
# -----
results_df = pd.DataFrame(results)
print("\nAll Model Results:")
display(results_df)

print("\nBest Model:")
print(best_model)

# -----
# SAVE BEST MODEL
# -----
import pickle
with open("best_model.pkl", "wb") as f:
    pickle.dump(best_model, f)

print("\nBest model saved as best_model.pkl")

```

## Output:

All Model Results:

|   | model             | best_params                                       | accuracy | f1       | precision | recall   | roc_auc  |
|---|-------------------|---------------------------------------------------|----------|----------|-----------|----------|----------|
| 0 | logistic          | {'clf__C': 1}                                     | 0.973684 | 0.979021 | 0.972222  | 0.985915 | 0.997380 |
| 1 | random_forest     | {'clf__max_depth': None, 'clf__n_estimators': ... | 0.964912 | 0.972222 | 0.958904  | 0.985915 | 0.995578 |
| 2 | gradient_boosting | {'clf__learning_rate': 0.2, 'clf__n_estimators... | 0.956140 | 0.965035 | 0.958333  | 0.971831 | 0.994759 |

Best Model:

```
Pipeline(steps=[('scaler', StandardScaler()),
                  ('clf', LogisticRegression(C=1, max_iter=1000))])
```

Best model saved as best\_model.pkl

## Practical No.: 11

**Aim:** To use advanced optimization techniques like Evolutionary algorithms or Bayesian Optimization for Hyperparameter tuning.

### Theory:

- Hyperparameters control model behavior and require external tuning.
- Grid search and random search are simple but inefficient for large spaces.
- Bayesian Optimization selects promising parameters using probabilistic modeling.
- Evolutionary algorithms evolve parameter sets through mutation and crossover.
- Advanced tuning improves accuracy and stability of deep learning systems.

**Requirements:** pip install numpy pandas scipy scikit-learn torch torchvision torchaudio bayesian-optimization matplotlib

### Code:

```
# -----  
# FINAL BAYESIAN OPTIMIZATION SCRIPT  
# -----  
  
import time  
import warnings  
warnings.filterwarnings("ignore", category=RuntimeWarning)  
  
try:  
    from bayes_opt import BayesianOptimization  
except:  
    raise ImportError("Install first: pip install bayesian-  
optimization")  
  
# -----  
# INSERT YOUR REAL evaluate_network() FUNCTION HERE  
# -----  
# This function MUST accept these parameters:  
# dropout, learning_rate, neuronPct, neuronShrink  
# And MUST return a single scalar SCORE to MAXIMIZE.  
# In your PDF, they return negative loss, which is correct.  
# -----  
  
def evaluate_network(dropout, learning_rate, neuronPct,  
neuronShrink):  
    """  
    IMPORTANT:  
    Replace this dummy block with your actual
```

```

evaluate_network
    from the PDF. Ensure the function RETURNS a numeric
    score.
    """

    # --- DELETE BELOW (dummy example) ---
    import math
    loss = (
        (dropout - 0.2)**2 * 50 +
        (learning_rate - 0.01)**2 * 500 +
        (neuronPct - 0.25)**2 * 20 +
        (neuronShrink - 0.3)**2 * 20 +
        0.05*math.sin(4*neuronPct)
    )
    return -loss # BayesOpt maximizes the score
    # --- DELETE ABOVE ---

# -----
# Parameter bounds (EXACTLY from your PDF)
# -----
pbounds = {
    "dropout": (0.0, 0.499),
    "learning_rate": (1e-6, 0.1),
    "neuronPct": (0.01, 1),
    "neuronShrink": (0.01, 1)
}

# -----
# Create optimizer
# -----
optimizer = BayesianOptimization(
    f=evaluate_network,
    pbounds=pbounds,
    verbose=2,          # full output
    random_state=1,
)

# -----
# Run optimization
# -----
print("\n🚀 Starting Bayesian Optimization...")
start_time = time.time()

optimizer.maximize(
    init_points=10,      # random exploration
    n_iter=20            # Bayesian optimization iterations
)

elapsed = time.time() - start_time

# -----

```

```
# Print results
# -----
print("\n🕒 Total Runtime: {:.2f} seconds".format(elapsed))
print("💧 BEST RESULT FOUND:")
print(optimizer.max)

best_params = optimizer.max["params"]
best_score = optimizer.max["target"]

print("\nBest Score :", best_score)
print("Best Params:", best_params)
```

## Output:

(.venv) PS E:\Hemlata\_MSc\_IT\Academics\SEM-III\AAI\AAI Practical\AAIPracticalsCode> python aaip11.py

🚀 Starting Bayesian Optimization...

| iter | target    | dropout   | learn...  | neuronPct | neuron... |
|------|-----------|-----------|-----------|-----------|-----------|
| 1    | -3.081973 | 0.2080939 | 0.0720327 | 0.0101132 | 0.3093092 |
| 2    | -0.955018 | 0.0732311 | 0.0092347 | 0.1943976 | 0.3521051 |
| 3    | -4.641707 | 0.1979869 | 0.0538821 | 0.4250025 | 0.6883673 |
| 4    | -7.215110 | 0.1020216 | 0.0878118 | 0.0371137 | 0.6737628 |
| 5    | -1.463812 | 0.2082350 | 0.0558694 | 0.1489830 | 0.2061204 |
| 6    | -9.034377 | 0.3995715 | 0.0968261 | 0.3202899 | 0.6953993 |
| 7    | -7.740304 | 0.4373181 | 0.0894607 | 0.0941937 | 0.0486642 |
| 8    | -4.441497 | 0.0847453 | 0.0878143 | 0.1073633 | 0.4268965 |
| 9    | -8.789470 | 0.4779868 | 0.0533169 | 0.6949583 | 0.3223604 |
| 10   | -8.802675 | 0.3425639 | 0.0834627 | 0.0281053 | 0.7526428 |
| 11   | -3.310434 | 0.2234988 | 0.0240531 | 0.1739215 | 0.6896328 |
| 12   | -5.493756 | 0.0901788 | 0.0780425 | 0.6057791 | 0.3238128 |
| 13   | -3.094703 | 0.1310730 | 0.0260040 | 0.6090464 | 0.2230624 |
| 14   | -0.837425 | 0.1018250 | 1e-06     | 0.1431879 | 0.2498773 |
| 15   | -8.143921 | 0.3700184 | 0.0932365 | 0.6349238 | 0.1898162 |
| 16   | -0.909591 | 0.0945513 | 1e-06     | 0.3240342 | 0.2145941 |
| 17   | -4.827361 | 0.0       | 1e-06     | 0.4834200 | 0.0135624 |
| 18   | -0.158506 | 0.2223339 | 1e-06     | 0.2704832 | 0.3393963 |
| 19   | -10.58959 | 0.3555669 | 0.0295502 | 0.3411860 | 0.9698208 |
| 20   | -14.94414 | 0.0       | 1e-06     | 1.0       | 0.01      |
| 21   | -0.107945 | 0.2201567 | 0.0070076 | 0.2625770 | 0.3427852 |
| 22   | -23.06214 | 0.0       | 1e-06     | 1.0       | 1.0       |
| 23   | -4.885989 | 0.0       | 1e-06     | 0.01      | 0.01      |

|    |           |           |       |           |           |
|----|-----------|-----------|-------|-----------|-----------|
| 23 | -4.885989 | 0.0       | 1e-06 | 0.01      | 0.01      |
| 24 | -4.477206 | 0.0       | 1e-06 | 0.3206100 | 0.6376066 |
| 25 | -1.296666 | 0.3164863 | 1e-06 | 0.1253005 | 0.4079815 |
| 26 | -0.895652 | 0.1895335 | 1e-06 | 0.2816962 | 0.4968426 |
| 27 | -13.00398 | 0.0       | 1e-06 | 0.01      | 1.0       |
| 28 | -0.268910 | 0.1959632 | 1e-06 | 0.1641039 | 0.3447375 |
| 29 | -0.848549 | 0.1047698 | 1e-06 | 0.3568873 | 0.3579373 |
| 30 | -6.320914 | 0.0       | 0.1   | 0.2502472 | 0.1930389 |

🕒 Total Runtime: 2.36 seconds

💧 BEST RESULT FOUND:

{'target': np.float64(-0.10794585279172497), 'params': {'dropout': np.float64(0.22015673407517306), 'learning\_rate': np.float64(0.0070076394208083695), 'neuronPct': np.float64(0.2625770256287745), 'neuronShrink': np.float64(0.34278527176843804)}}

Best Score : -0.10794585279172497

Best Params: {'dropout': np.float64(0.22015673407517306), 'learning\_rate': np.float64(0.0070076394208083695), 'neuronPct': np.float64(0.2625770256287745), 'neuronShrink': np.float64(0.34278527176843804)}

## Practical No.: 12

**Aim:** To deploy a Machine Learning model in a production environment using Containerization and Cloud services.

### Theory:

- Deployment makes an ML model available for real-world prediction through APIs.
- Docker containers package the model and dependencies into a portable environment.
- Kubernetes orchestrates containers for scalability and load balancing.
- Cloud platforms (AWS, GCP, Azure) host and serve models reliably.
- CI/CD pipelines allow continuous updates, monitoring, and maintenance.

**Requirements:** VS-Code, Docker, Google Colab

### Code & Outputs:

#### Step-1: Open Google Colab

Create a new notebook in Google Drive.

#### Step-2: Train a Machine Learning Model in Colab

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import pickle

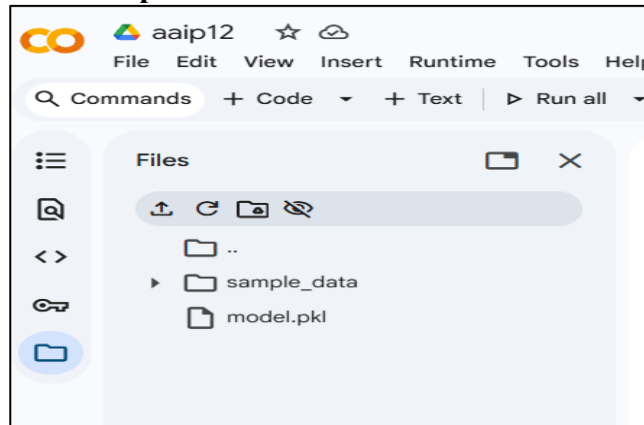
# load data
iris = load_iris()
X = iris.data
y = iris.target

# train model
model = LogisticRegression(max_iter=1000)
model.fit(X, y)

# save model
with open("model.pkl", "wb") as f:
    pickle.dump(model, f)

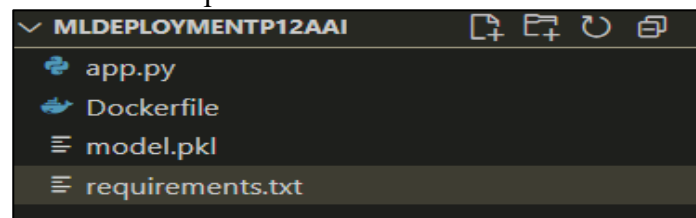
print("model.pkl saved!")
```

Model saved as model.pkl

**Step-3: Download the model.pkl File****Step-4: Open VS Code and Create a Project Folder**

Name it as *MLDeploymentP12AAI* (Can name as per your choice)

We'll create the files in further steps.

**Step-5: Copy model.pkl into the Folder****Step-6: Create app.py in VS Code and paste the API code**

```
from flask import Flask, request, jsonify
import pickle
import numpy as np

app = Flask(__name__)

# Load model
with open("model.pkl", "rb") as f:
    model = pickle.load(f)

@app.route("/", methods=["GET"])
def home():
    return "API is running!"

@app.route("/predict", methods=["POST"])
def predict():
    data = request.json["data"]
    data = np.array(data).reshape(1, -1)
    pred = model.predict(data)[0]
    return jsonify({"prediction": int(pred)})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

**Step-7: Create requirements.txt and add the below lines**

```
flask  
numpy  
scikit-learn
```

**Step-8: Create the Dockerfile**

```
FROM python:3.10-slim  
WORKDIR /app  
COPY . .  
RUN pip install -r requirements.txt  
EXPOSE 5000  
CMD ["python", "app.py"]
```

**Step-9: Install Docker Desktop**

Install Docker for Windows x64 (AMD/Intel).

<https://www.docker.com/products/docker-desktop/>

Also install WSL2 Linux Kernel from:

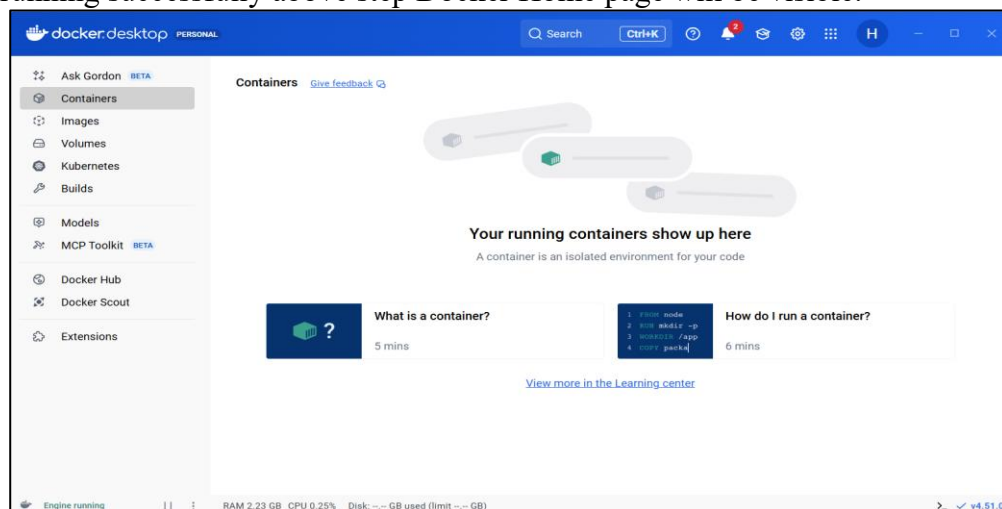
[https://wslstorestorage.blob.core.windows.net/wslblob/wsl\\_update\\_x64.msi](https://wslstorestorage.blob.core.windows.net/wslblob/wsl_update_x64.msi)

Then run in cmd:

```
wsl --update  
wsl --status
```

```
C:\Windows\System32>wsl --update  
Checking for updates.  
The most recent version of Windows Subsystem for Linux is already installed.  
  
C:\Windows\System32>wsl --status  
Default Version: 2  
  
C:\Windows\System32>
```

After running successfully above step Docker Home page will be visible.



**Step-10: Build & Run the Docker Container**

Open VS Code Terminal : *Ctrl + ~*

Enter the below command To Build Docker Image:

*docker build -t iris-api .*

```

PS E:\Hemlata_MSc_IT\Academics\SEM-III\AAI\AAI Practical\MLDeploymentP12AAI> docker build -t iris-api .
[+] Building 102.9s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring Dockerfile: 158B
=> [internal] load metadata for docker.io/library/python:3.10-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/python:3.10-slim@sha256:abca1478399019424da6d9f37888ea2842d82d624f2ca78078c92b1a30d2fa7b
=> => resolve docker.io/library/python:3.10-slim@sha256:abca1478399019424da6d9f37888ea2842d82d624f2ca78078c92b1a30d2fa7b
=> => sha256:9793c1b1e51abab0971146499e99c7114b8b8f9238022d6d6970c9317b02 / 1.29MB
=> => sha256:683c365981e6b36ff123a6fdd5891f83b363fa0954683b1de7962d15931e / 13.82MB
=> => sha256:f86b98c4d0f9c7caf377d5426b6c24a4c45058d795fd1a06c1a7c54b626c084 / 250B
=> => sha256:0e4bc2bd656e0e04e3c743af70e5650bac2258243eb0949dea51c8b7863db / 29.78MB
=> => extracting sha256:0e4bc2bd656e0e04e3c743af70e5650bac2258243eb0949dea51c8b7863db
=> => extracting sha256:9793c1b1e51abab0971146499e99c7114b8b8f9238022d6d6970c9317b02
=> => extracting sha256:683c365981e6b36ff123a6fdd5891f83b363fa0954683b1de7962d15931e
=> => extracting sha256:f86b98c4d0f9c7caf377d5426b6c24a4c45058d795fd1a06c1a7c54b626c084
=> [internal] load build context
=> => transferring context: 1.68kB
=> [2/4] WORKDIR /app
=> [3/4] COPY . .
=> [4/4] RUN pip install -r requirements.txt
=> exporting to image
=> exporting layers
=> exporting manifest sha256:b074166b366f3fcd95a128d15f8d2bc53691155a7d956e76d3f1683b70e4
=> exporting config sha256:7f65e3101d56611d42157e8d38b73d346d0c49854a5e221999c4dd1f36f4
=> exporting attestation manifest sha256:4b08337c216a1d864628e98057ec49fd26f6b65263dfce527c7b2eacade4dc
=> exporting manifest list sha256:afa67c442fd26732A4f480f1f320aaa97bc90859198465c03fc7666d71a2746f
=> naming to docker.io/library/iris-api:latest
=> unpacking to docker.io/library/iris-api:latest
PS E:\Hemlata_MSc_IT\Academics\SEM-III\AAI\AAI Practical\MLDeploymentP12AAI>
  
```

Enter the below command To Run Docker Container:

*docker run -d -p 5000:5000 iris-api*

```

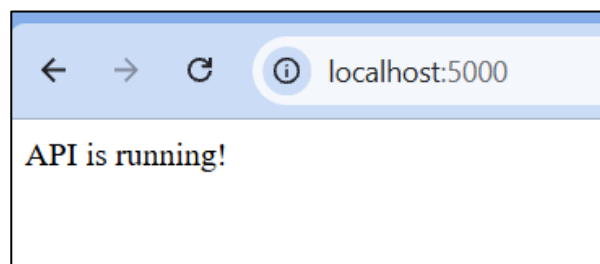
PS E:\Hemlata_MSc_IT\Academics\SEM-III\AAI\AAI Practical\MLDeploymentP12AAI> docker run -d -p 5000:5000 iris-api
b7bf3f4d3e3ebfcab7e6d3a0dd91bc9c11f40d8d1a3ac8c9afd8c626d6410f7
PS E:\Hemlata_MSc_IT\Academics\SEM-III\AAI\AAI Practical\MLDeploymentP12AAI>
  
```

Give permission if Windows asks → Click **Allow**.

**Step-11: Test the API**

Open any Browser & Go to: <http://localhost:5000/>

If you see **API is running!** Then your docker is working properly.

**Test Prediction (in CMD)**

Paste:

```

curl -X POST http://localhost:5000/predict ^
-H "Content-Type: application/json" ^
-d '{"data": [5.1, 3.5, 1.4, 0.2]}'
  
```



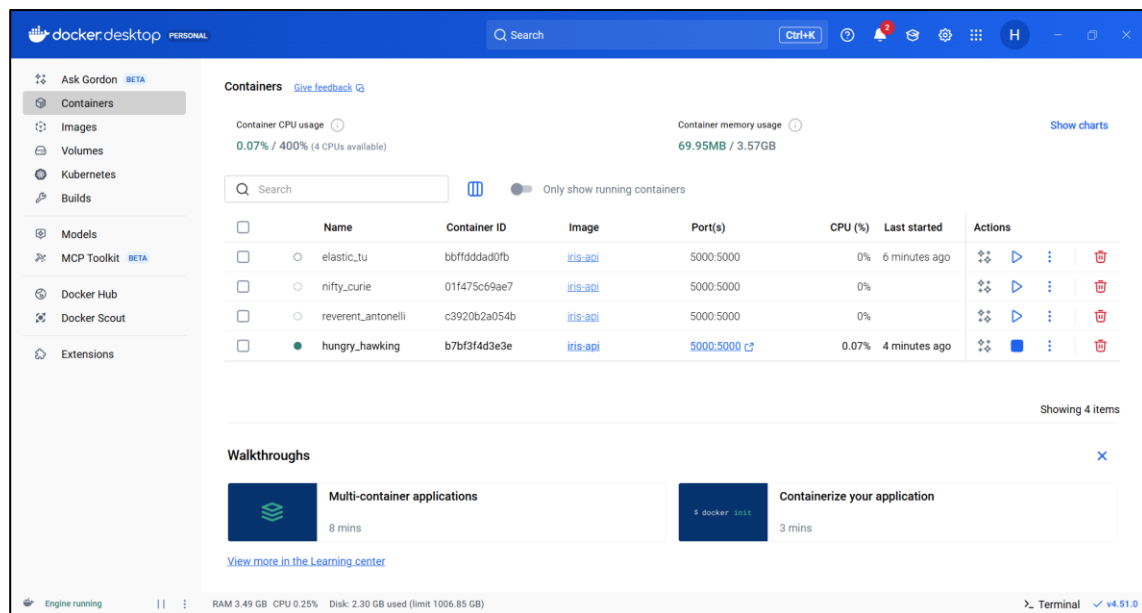
**Output:**

```
{"prediction": 0}
```

```
C:\Users\Hemlata>curl -X POST http://localhost:5000/predict ^
More? -H "Content-Type: application/json" ^
More? -d "{\"data\": [5.1, 3.5, 1.4, 0.2]}"
{"prediction":0}

C:\Users\Hemlata>|
```

Your Docker will look like after all successful steps.

**Conclusion:**

In this practical, a Machine Learning model was:

1. Trained in Google Colab
2. Exported as model.pkl
3. Converted into an API using Flask
4. Containerized using Docker (Dockerfile)
5. Deployed locally on port 5000
6. Successfully tested with a POST request

This demonstrates a full ML deployment workflow used in real-world production systems.

This practical demonstrates the **complete ML deployment pipeline**:

**Train → Save → Build API → Dockerize → Deploy**

You transformed a machine learning model into a **production-ready API** using:

- **Flask** → for the web API
- **Docker** → for containerization
- **Local or Cloud** → for deployment

This is exactly what companies do for production systems.

## Practical No.: 13

**Aim:** To use Python libraries such as GPT-2 or Textgram to train Generative models on a Corpus of text data and Generate new text based on the patterns it has learned.

### Theory:

- Generative language models learn statistical patterns in text to produce new sentences.
- GPT-2 uses transformer architecture to predict the next token based on context.
- Training involves tokenization and minimizing prediction error on text sequences.
- Sampling techniques like temperature, top-k, and top-p control creativity.
- Used for article writing, chatbots, storytelling, and automated content creation.

**Requirements:** !pip install textgenrnn

!pip install transformers datasets accelerate torch

### Code & Output:

#### Step-1: Upload dataset

```
from google.colab import files
uploaded = files.upload()
```

#create a random text file and save it as data and mycorpus (as a .txt file) and upload it.

Choose Files 2 files

data.txt(text/plain) - 1189 bytes, last modified: 11/20/2025 - 100% done

mycorpus.txt(text/plain) - 3413 bytes, last modified: 11/20/2025 - 100% done

Saving data.txt to data.txt

Saving mycorpus.txt to mycorpus.txt

#### Step-2: Load tokenizer + model

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")
# GPT-2 does not have pad token → set pad token
tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
tokenizer_config.json: 100%  26.0/26.0 [00:00<00:00, 397B/s]
vocab.json: 100%  1.04M/1.04M [00:00<00:00, 10.3MB/s]
merges.txt: 100%  456k/456k [00:00<00:00, 10.1MB/s]
tokenizer.json: 100%  1.36M/1.36M [00:00<00:00, 16.3MB/s]
config.json: 100%  665/665 [00:00<00:00, 15.5kB/s]
model.safetensors: 100%  548M/548M [00:07<00:00, 61.8MB/s]
generation_config.json: 100%  124/124 [00:00<00:00, 3.50kB/s]

```

### **Step-3: Prepare dataset**

```

from datasets import load_dataset

# Load either file

#dataset = load_dataset("text", data_files={"train":
"data.txt"})

dataset = load_dataset("text", data_files={"train":
"mycorpus.txt"})

```

Generating train split:  11/0 [00:00<00:00, 333.21 examples/s]

### **Step-4: Tokenize dataset**

```

def tokenize(batch):
    return tokenizer(
        batch["text"],
        truncation=True,
        padding="max_length",
        max_length=128 )

# Tokenize

tokenized_dataset = dataset.map(tokenize, batched=True,
remove_columns=["text"])

# Add labels for GPT2 training

tokenized_dataset = tokenized_dataset.map(
    lambda x: {"labels": x["input_ids"]},
    batched=True)

tokenized_dataset.set_format("torch")

```

Map: 100%  11/11 [00:00<00:00, 232.80 examples/s]  
Map: 100%  11/11 [00:00<00:00, 284.39 examples/s]

**Step-5: Training**

```
import os

os.environ["WANDB_DISABLED"] = "true"    # Avoid wandb login
prompt

from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="./gpt2_finetuned",
    per_device_train_batch_size=2,
    num_train_epochs=3,
    logging_steps=50,
    save_steps=200,
    report_to="none"    # Disable all online logging)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],)

trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:666: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used
warnings.warn(warn_msg)
'loss_type=None' was set in the config but it is unrecognized. Using the default loss: 'ForCausalMLoss'.
[18/18 01:38, Epoch 3/3]

Step Training Loss
TrainOutput(global_step=18, training_loss=3.524759716457791, metrics={'train_runtime': 104.44, 'train_samples_per_second': 0.316, 'train_steps_per_second': 0.172, 'total_flos': 2155659264000.0,
'train_loss': 3.524759716457791, 'epoch': 3.0})
```

**Step-6: Generate Text**

```
prompt = "Artificial intelligence is"

input_ids = tokenizer.encode(prompt, return_tensors="pt")

output = model.generate(
    input_ids,
    max_length=120,
    temperature=0.8,
    top_p=0.9,
    do_sample=True)

print(tokenizer.decode(output[0], skip_special_tokens=True))
```

```
The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's 'attention_mask' to obtain reliable results.
Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observe unexpected behavior. Please pass your input's 'attention_mask' to obtain reliable results.
Artificial intelligence is a system designed to analyze data to make it more efficient. AI makes smarter decisions, improves learning and learns more.
```

## Practical No.: 14

**Aim:** To experiment with Neural Networks like Generative Adversarial Networks (GANs) using Python libraries like TensorFlow or PyTorch to generate new images based on a dataset of images.

### Theory:

- GANs train generator and discriminator networks adversarially to create realistic images.
- Generator maps random noise to fake images, while discriminator classifies real vs fake.
- Variants like DCGAN, StyleGAN, and CycleGAN enhance image quality.
- GANs generate faces, artwork, product designs, and deepfake media.
- Widely used in fashion, entertainment, design, and image restoration.

**Requirements:** !pip install tensorflow matplotlib pillow imageio

### Code:

```
import tensorflow as tf
import glob
import imageio
import matplotlib.pyplot as plt
import numpy as np
import os
import PIL
import time
from tensorflow.keras import layers
from IPython import display
# --- Load and prepare the dataset ---
(train_images, _), (_, _) =
tf.keras.datasets.mnist.load_data()
train_images = train_images.reshape(train_images.shape[0],
28, 28, 1).astype('float32')
train_images = (train_images - 127.5) / 127.5 # Normalize
to [-1, 1]
BUFFER_SIZE = 60000
BATCH_SIZE = 256

# Batch and shuffle the data
train_dataset = (
    tf.data.Dataset.from_tensor_slices(train_images)
    .shuffle(BUFFER_SIZE)
    .batch(BATCH_SIZE))

# --- Generator model ---
def make_generator_model():
    model = tf.keras.Sequential()
```

```

        model.add(layers.Dense(7 * 7 * 256, use_bias=False,
input_shape=(100,)))
        model.add(layers.BatchNormalization())
        model.add(layers.LeakyReLU())
        model.add(layers.Reshape((7, 7, 256)))
        assert model.output_shape == (None, 7, 7, 256)
        model.add(layers.Conv2DTranspose(128, (5, 5),
strides=(1, 1),
padding='same',
use_bias=False))
        model.add(layers.BatchNormalization())
        model.add(layers.LeakyReLU())
        model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2,
2),
padding='same',
use_bias=False))
        model.add(layers.BatchNormalization())
        model.add(layers.LeakyReLU())
        model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2,
2),
padding='same',
use_bias=False,
activation='tanh'))
        assert model.output_shape == (None, 28, 28, 1)
        return model

generator = make_generator_model()
noise = tf.random.normal([1, 100])
generated_image = generator(noise, training=False)
plt.imshow(generated_image[0, :, :, 0], cmap='gray')

# --- Discriminator model ---
def make_discriminator_model():
    model = tf.keras.Sequential()
    model.add(layers.Conv2D(64, (5, 5), strides=(2, 2),
padding='same', input_shape=[28,
28, 1]))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))
    model.add(layers.Conv2D(128, (5, 5), strides=(2, 2),
padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))
    model.add(layers.Flatten())
    model.add(layers.Dense(1))
    return model
discriminator = make_discriminator_model()
decision = discriminator(generated_image)
print(decision)

```

```
# --- Loss functions ---
cross_entropy =
tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output),
real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output),
fake_output)
    total_loss = real_loss + fake_loss
    return total_loss
def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output),
fake_output)

# --- Optimizers ---
generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

# --- Checkpoints ---
checkpoint_dir = './training_checkpoints'
checkpoint_prefix = os.path.join(checkpoint_dir, "ckpt")
checkpoint =
tf.train.Checkpoint(generator_optimizer=generator_optimizer,
discriminator_optimizer=discriminator_optimizer,
generator=generator,
discriminator=discriminator)

# --- Training ---
EPOCHS = 10
noise_dim = 100
num_examples_to_generate = 16
seed = tf.random.normal([num_examples_to_generate,
noise_dim])
@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, noise_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as
disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images,
training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output,
fake_output)
```

```
        gradients_of_generator = gen_tape.gradient(gen_loss,
generator.trainable_variables)
        gradients_of_discriminator =
disc_tape.gradient(disc_loss,
discriminator.trainable_variables)

generator_optimizer.apply_gradients(zip(gradients_of_generat
or, generator.trainable_variables))

discriminator_optimizer.apply_gradients(zip(gradients_of_dis
criminator, discriminator.trainable_variables))

def generate_and_save_images(model, epoch, test_input):
    predictions = model(test_input, training=False)

    fig = plt.figure(figsize=(4, 4))
    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i + 1)
        plt.imshow((predictions[i, :, :, 0] + 1) / 2.0,
cmap='gray')
        plt.axis('off')

    plt.savefig('image_at_epoch_{:04d}.png'.format(epoch))
    plt.close(fig)

def train(dataset, epochs):
    for epoch in range(epochs):
        start = time.time()

        for image_batch in dataset:
            train_step(image_batch)

        display.clear_output(wait=True)
        generate_and_save_images(generator, epoch + 1, seed)

        if (epoch + 1) % 15 == 0:
            checkpoint.save(file_prefix=checkpoint_prefix)

        print(f'Time for epoch {epoch + 1} is {time.time() -
start:.2f} sec')

        display.clear_output(wait=True)
        generate_and_save_images(generator, epochs, seed)

# --- Run training ---
train(train_dataset, EPOCHS)
# --- Restore latest checkpoint ---
checkpoint.restore(tf.train.latest_checkpoint(checkpoint_dir
))
```



```
# --- Create a GIF ---
def display_image(epoch_no):
    return
PIL.Image.open(f'image_at_epoch_{epoch_no:04d}.png')

display_image(EPOCHS)

anim_file = 'dcgan.gif'

with imageio.get_writer(anim_file, mode='I') as writer:
    filenames = sorted(glob.glob('image_at_epoch_*.png'))
    for filename in filenames:
        image = imageio.imread(filename)
        writer.append_data(image)

print(f"Saved animation to {anim_file}")
```

**Output:**